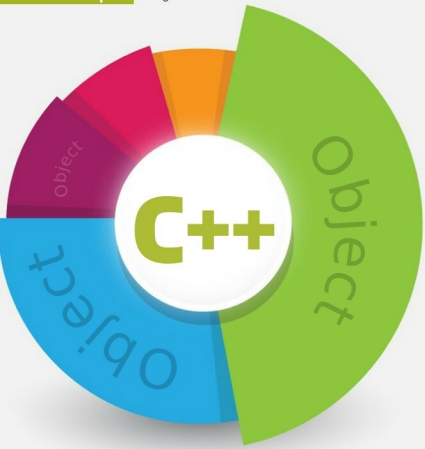

Understanding Object Slicing in C++

This article is intended for college students, as well as new and intermediate programmers. Although the idea of object slicing is known to many expert programmers, anyone who comes across this concept for the first time can fumble with the nitty-gritty. The article starts by introducing object slicing, before going on to show examples of how things can really go wrong. It ends by providing ways to avoid the problems.

In C++, it is possible to assign or copy a derived class object to the base class object (the other way around is not possible, though). But when you do that, you invariably run into something called object slicing. It means that when an object of a derived class is assigned or copied to the base class object, then the 'derived' portion of it is cut off or sliced, and only the 'base' portion is assigned to the base object. Thus the derived portion is never known to the base object. It is a very common glitch that many C++ programmers try to avoid and one can find discussions on it on the Internet.

The following example shows object slicing caused by copying a derived class object to the base class object (while using the pass-by-value):

```
#include <iostream>
using namespace std;

class Base
{
public:
Base(int a)
    {
      a_ = a;
    }

void print()
    {
cout<< "In Base::print() : a_ " << a_ <<endl;
    }

private:
int a_;
};

class Derived : public Base
{
public:
Derived(int a, int b):Base(a)
    {
      b_ = b;
```